

Abstract

論文首先提及目前有許多 workload 都是 memory-bound，memory-bound 會讓記憶體和 CPU 之間的 data 傳輸帶來明顯延遲和能耗上的問題，歸因於傳輸資料的 bus 大小有限且具有延遲。為了解決這種傳輸瓶頸的問題，研究者提出了 Processing-In-Memory (PIM) 的概念，為記憶體增加了處理能力，來緩解處理資料時須先經過傳輸的問題。不過先前的 PIM 研究主要依賴模擬或簡化的硬體模型，缺乏真實硬體實驗。而這篇論文是針對第一個公開且商用地 PIM 系統 - UPMEM 進行評估，並提供了來自不同領域的 16 種 memory-bound workload，稱為 PrIM (Processing-In-Memory benchmarks)，最後對 UPMEM PIM 架構進行全面的特性分析與表現評估，旨在讓未來可能採用 PIM 系統的硬體和架構設計師提供建議和參考。

Introduction

UPMEM 採用 2D DRAM 並結合處理器，稱為 DRAM Processing Units (DPUs)，其 DPU 中的 thread 可以採非同步的方式運行，並且能支援多種運算與 data type。論文首先測試了 DPU 在不同算術運算與 data type 得到的 throughput，並測量 DPU 中兩種記憶體空間的 bandwidth (MRAM 和 WRAM)，以及測試主記憶體和 MRAM 對於不同 data type 和大小的 bandwidth。

Upmem PIM

論文首先介紹了 UPMEM 本身的基本架構，例如 64MB 的 DRAM 區塊(稱為主記憶體 MRAM)、24KB 的指令記憶體(IRAM)以及 64KB 的暫存記憶體(WRAM)。主機和 DPU 之間會以 MRAM 進行資料傳輸，而 UPMEM 的 SDK 提供了函式庫能進行序列傳輸與平行傳輸等等。接著便介紹了一些 14-stage 指令 pipeline、MRAM 和 WRAM 之間的 bandwidth 最高可達 800MB/s 等等的細項內容。

另外文中提到了對 UPMEM 系統 programming 的建議：

1. 盡量長時間執行 DPU 的 parallel code，來避免頻繁和 CPU 互動，減少記憶體 bus 的資料傳輸。
2. 將 workload 分成多個獨立的 data blocks，來充分利用 DPU 平行運算的優勢，同時避免 DPU 之間的通信傳輸(因為 DPU 之間的資料傳輸需要經過 CPU)
3. 盡可能地使用系統中的 DPU。
4. 每個 DPU 至少啟用 11 個 tasklets(可以當成 thread，這與前面提到的 14-stage 指令 pipeline 有關，要至少 11 個 thread 才能讓 DPU 保持運算狀態)

PERFORMANCE CHARACTERIZATION

在介紹完 UPMEM 架構後，論文針對以下內容進行了測試：

1. 測試 DPU 算術運算的 throughput 和 WRAM bandwidth
2. MRAM 和 WRAM 的 sustained bandwidth

3. workload 的算術密集度對於 DPU 算術運算的 throughput 影響
4. host 主記憶體和 MRAM 區塊之間的 bandwidth

由於論文做的實驗非常多，沒辦法全部整理實驗細節與結果(篇幅太長)，因此這裡整理了一些論文在經過 microbenchmarks 後得到的 key observation：

1. 算術運算在 DPU 使用 11 個以上的 tasklets 時會達到飽和。
2. DPU 對 32-bits 與 64-bits 的整數加法與減法有提供硬體支援，而對於除法、乘法以及浮點數操作則沒有，只能由 runtime library 來模擬，throughput 會大幅降低。
3. WRAM 提供的 sustained throughput 與存取模式無關；另外 8-byte WRAM 的存取操作僅需 1 cycle。
4. MRAM 存取延遲會隨著傳輸數據的大小呈線性增加；MRAM 的最大理論 throughput 為 2 bytes per cycle.
5. DPU 本質上是 compute-bound processor (在 low operational intensity 下算術運算的 throughput 就達到最高值了)
6. CPU 和 DPU 之間的傳輸 bandwidth 會隨著 rank 中 DPU 數量的增加而提升

根據以上的觀察結果，論文提出了一些 programming 上的建議：

1. MRAM 和 WRAM 之間需要傳輸數據時，盡可能以大的 DRAM 傳輸資料量來最大化 MRAM throughput。
2. 對於 MRAM 和 WRAM 之間的小型傳輸，可以選擇比實際需求更多的 bytes 數進行傳輸，從而增加後續在 WRAM 中找到所需數據的可能性，降低使用的 MRAM 次數。
3. 根據 WRAM 的使用量，去平衡 MRAM 和 WRAM 之間的傳輸大小，

PRIM BENCHMARKS

這部分對應論文的第四章(IV. PRIM BENCHMARKS)，裡面主要在講述其所提供的 benchmarks 之特性(記憶體存取模式、計算操作和 data type 等等)。這些 benchmarks 所採用的 workload 在使用主機 CPU 進行測試後，會發現結果處於 memory-bound 區塊，表明這些 workload 都是受限於記憶體的 bandwidth，而非 CPU 的計算效能，因此很適合用於 PIM 效能評估。

最重要的是，作者也針對這些資料運算情境(或是說演算法)提出了 UPMEM PIM 的優化方案。不僅能夠讓讀者初步了解 UPMEM 的使用與程式設計思路，也讓我體會到其與一般 CPU 不同的魅力，例如作者是如何發揮 UPMEM 的多執行緒(tasklets)以及多顆 DPU 存取的優勢，來實現快速向量相加、矩陣乘法、廣度優先搜尋(BFS)、Needleman-Wunsch (NW)算法等等，能夠讓讀者避免一些 UPMEM 的使用誤區。

而後作者利用這些 benchmarks 對 UPMEM 進行了評估，主要是變化一些諸如單一 Rank 中 DPU 採用的數量(1、4、16、64 顆 DPU)，或是 Rank 的數量(4、8、16、32 個 Rank)來去比較效能，或是和 CPU、GPU 進行對比。

這裡整理了作者針對實驗觀察所得到的四項結論：

1. UPMEM 本質上是 Compute-bound，大部分適用的 workloads 為 Memory-bound 的，在使用 UPMEM 時需要改變對 compute 和 memory access 的思考方式，因為 UPMEM 在執行這兩項任務的成本與主流 CPU 恰好相反。
2. UPMEM 不適合處理複雜的算術運算，因此 **UPMEM 適合處理僅包含位元運算或整數加減法的 workloads**。這種類型的 workload 經測試後甚至能比 GPU 快大約 2.54 倍。
3. 由於 UPMEM 內部 DPU 交流需要依靠 CPU 負責處理，**UPMEM 更適合處理那些幾乎不需要 DPU 交換資訊的 workloads**，否則對效能有巨大影響。
4. UPMEM 和 CPU 相比，不論在效能和能耗表現上都更有優勢(memory-bound workloads 的情境)。另外有 10 個 workload 表現比 GPU 更好，這些 workload 的特性包含：streaming memory accesses、幾乎沒有 inter-DPU 交流、幾乎不使用乘除或浮點運算等等。

My thoughts

這篇論文做的實驗太詳細了，能夠讓讀者很好地去認識 UPMEM PIM 架構的特性，並且作者對所有的實驗觀察結果都產生了一些 programming 上的建議，對於正在研究 UPMEM 的學者有很大的幫助。另外這些實驗都是利用真實硬體進行分析，有很高的參考價值，公開的測試基準與工具也對研究者有很大的幫助。

由於我本身在研究的議題是 DNA 上的 in-memory-searching，當時發現 GRIM-Filter 演算法可以改良成僅需位元運算與簡單整數加減法來實現差不多的效果，但在 IMS Nand flash SSD 上的表現不是特別好。而這篇論文提到 UPMEM 很適合那些僅包含位元運算或整數加減法的 workload，或許 UPMEM 在這份想法上是一個不錯的方案，前提是資料能夠塞得進 UPMEM 裡。

論文裡有些內容暫時沒有理解得很透徹，但幫助甚大，特別是 UPMEM 的架構和一些 UPMEM 特性在 programming 上的建議，我想日後等我真正確定要將想法實現在 UPMEM PIM 架構上，肯定會再回來學習這篇論文。